



UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

COMPUTAÇÃO DE ALTO DESEMPENHO

DOCENTE: CARLA DOS SANTOS SANTANA

E SAMUEL XAVIER DE SOUZA

DISCENTE: CLARA VIRGÍNIA NASCIMENTO SANTOS (20240002212)

Tarefa 10

1. ENUNCIADO

Implemente novamente o estimador da tarefa 8 que usa um contador compartilhado e o `rand_r` substituindo o `#pragma omp critical` pelo `#pragma omp atomic`. Compare essas duas implementações com suas versões que usam contadores privados. Agora, compare essas com uma 5ª versão que utiliza apenas a cláusula `reduction` ao invés das diretivas de sincronização. Reflita sobre a aplicabilidade desses mecanismos em termos de desempenho e produtividade e proponha um roteiro para quando utilizar qual mecanismo de sincronização, incluindo `critical` nomeadas e `locks` explícitos.

2. DISCUSSÃO

Nas análises anteriores, foi observado que, ao paralelizar um laço com a OpenMP, surge a chamada `race condition`, isto é, quando múltiplas threads tentam acessar e modificar simultaneamente uma mesma variável compartilhada, gerando um resultado dependente da ordem exata em que cada thread executa. Dessa maneira, para garantir a corretude do programa, é necessário utilizar algum mecanismo de sincronização. Contudo, cada mecanismo tem um custo de desempenho diferente, e o objetivo desta tarefa é, sobretudo, comparar esses custos.

2.1. Mecanismos de sincronização

A OpenMP oferece diferentes formas de proteger uma variável compartilhada, a saber:

- `#pragma omp critical`: cria uma região onde apenas uma thread por vez pode entrar e executar o bloco. É bastante flexível, já que protege qualquer conjunto

de instruções; todavia, é o mecanismo mais pesado, pois internamente utiliza um lock (mutex) e força as threads a esperarem em fila.

- `#pragma omp atomic`: protege uma única operação de leitura-modificação-escrita sobre uma variável escalar (como `count++` ou `count += x`). Diferentemente do `critical`, ele é traduzido para uma instrução atômica diretamente no hardware, sendo, por isso, mais leve. No entanto, só funciona para operações simples.
- `reduction(+:count)`: ao invés de o programador inserir uma diretiva de sincronização, o próprio OpenMP fornece a cada thread uma cópia local da variável (inicializada com o elemento neutro, ou seja, 0 para a soma); cada thread acumula na sua cópia e, ao final, a OpenMP combina automaticamente todas as cópias na variável original. Dessa forma, praticamente não há contenção entre as threads durante o laço.
- `critical` nomeada (`#pragma omp critical(nome)`): por padrão, todas as regiões `critical` sem nome são mutuamente exclusivas entre si, isto é, mesmo protegendo dados diferentes, elas concorrem pelo mesmo lock global. Ao nomear o `critical`, somente as regiões com o mesmo nome se bloqueiam mutuamente, ao passo que regiões com nomes diferentes podem executar simultaneamente. Assim, é possível proteger dados independentes sem serializar desnecessariamente o programa.
- locks explícitos (`omp_lock_t`): com as funções `omp_init_lock`, `omp_set_lock`, `omp_unset_lock` e `omp_destroy_lock`, o programador controla manualmente o bloqueio. É a abordagem mais flexível, pois permite, por exemplo, ter um lock por elemento de um vetor ou travar e destravar em pontos diferentes do código. Contudo, exige gerenciar o ciclo de vida do lock e traz o risco de deadlocks.

2.2. Contador compartilhado vs. contador privado

Um ponto central desta tarefa é diferenciar onde a sincronização acontece:

- Contador compartilhado: a variável `count` é incrementada dentro do laço a cada ponto que cai no círculo. Por consequência, a sincronização (`critical` ou `atomic`) é acionada milhões de vezes, gerando enorme contenção entre as threads.
- Contador privado: cada thread acumula em uma variável local (`count_local`) sem nenhuma sincronização durante o laço; somente ao final, uma única vez por thread, esse valor local é somado ao contador compartilhado. Dessa

forma, o número de sincronizações cai de milhões para apenas o número de threads.

2.3. rand_r e segurança em paralelo

Tal qual visto nas tarefas anteriores, a função rand() mantém um estado interno global, e por isso ela não é segura para uso em paralelo, já que as threads disputariam esse estado. Dessa maneira, foi utilizada a função rand_r, que recebe um ponteiro para uma seed própria de cada thread. Assim, cada thread gera sua sequência de números aleatórios de forma independente, sem condição de corrida na geração dos pontos.

2.4. ETAPAS PARA RESOLUÇÃO DA TAREFA

Etapas para a resolução da tarefa:

- 1º) Implementar a Versão 1: contador compartilhado + #pragma omp critical;
- 2º) Implementar a Versão 2: contador compartilhado + #pragma omp atomic;
- 3º) Implementar a Versão 3: contador privado + critical (sincroniza só no final);
- 4º) Implementar a Versão 4: contador privado + atomic (sincroniza só no final);
- 5º) Implementar a Versão 5: cláusula reduction;
- 6º) Medir o tempo de cada versão com omp_get_wtime();
- 7º) Comparar desempenho e produtividade;
- 8º) Montar o roteiro de escolha do mecanismo.

2.5. COMPILAR

```
gcc -fopenmp tarefa10.c -o tarefa10 -lm
```

3. RESOLUÇÃO EM CÓDIGO DA TAREFA

Segue no apêndice.

4. OUTPUT

Segue no apêndice.

5. CONCLUSÃO

Os resultados evidenciam que o fator mais determinante para o desempenho não é qual diretiva de sincronização foi escolhida, mas sim com que frequência ela é acionada. As versões com contador compartilhado (Versões 1 e 2) sincronizam a cada iteração e, por isso, são drasticamente mais lentas, sendo que o critical se mostrou ainda pior que o atomic em razão do custo do lock. Por outro lado, as versões com contador privado (Versões 3 e 4) sincronizam apenas uma vez por thread, e por consequência apresentam ótimo desempenho, praticamente empatadas entre si. Por

fim, a cláusula reduction (Versão 5) une o melhor dos dois mundos, oferecendo tanto o melhor desempenho quanto a maior simplicidade de código. Dessa forma, conclui-se que, ao paralelizar, deve-se sempre buscar minimizar a sincronização dentro dos laços, recorrendo a mecanismos mais flexíveis como critical nomeadas e locks explícitos somente quando o problema realmente exigir.

6. APÊNDICE

8.1. RESOLUÇÃO EM CÓDIGO

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>
#include <time.h>

#define N 100000000 // muitos pontos pra estressar a sincronizacao

// gerador de aleatorios proprio (igual ao da tarefa 6) -> usado no lugar do
rand_r, já que é win
int meu_rand(unsigned int *seed) {
    *seed = *seed * 1103515245 + 12345;
    return (*seed >> 16) & 0x7fff; // retorna um numero de 0 ate 32767
}

// VERSAO 1: contador compartilhado + critical (essa eh a da tarefa 8)
void versao_compartilhado_critical() {
    double inicio = omp_get_wtime();
    long count = 0; // contador compartilhado entre todas as threads

    #pragma omp parallel
    {
        // cada thread tem sua propria seed -> evita race na geracao de
        aleatorios
        unsigned int seed = 12345 + omp_get_thread_num();

        #pragma omp for
        for (long i = 0; i < N; i++) {
            double x = (double)meu_rand(&seed) / 0x7fff; // ponto x entre 0 e
            double y = (double)meu_rand(&seed) / 0x7fff; // ponto y entre 0 e

            if (x*x + y*y <= 1.0) {
                #pragma omp critical // so uma thread por vez pode mexer no
                count
                {
                    count++;
                }
            }
        }
    }
}
```

```

    }
    }
}

double pi = (4.0 * count) / N;
double fim = omp_get_wtime();
printf("1) Compartilhado + critical -> pi: %.6f | Tempo: %fs\n", pi, fim -
inicio);
}

// VERSAO 2: contador compartilhado + atomic (trocando critical por atomic)
void versao_compartilhado_atomic() {
    double inicio = omp_get_wtime();
    long count = 0;

    #pragma omp parallel
    {
        unsigned int seed = 12345 + omp_get_thread_num();

        #pragma omp for
        for (long i = 0; i < N; i++) {
            double x = (double)meu_rand(&seed) / 0x7fff;
            double y = (double)meu_rand(&seed) / 0x7fff;
            if (x*x + y*y <= 1.0) {
                #pragma omp atomic // operacao atomica direto no hardware,
mais leve que o critical
                count++;
            }
        }
    }

    double pi = (4.0 * count) / N;
    double fim = omp_get_wtime();
    printf("2) Compartilhado + atomic -> pi: %.6f | Tempo: %fs\n", pi, fim -
inicio);
}

// VERSAO 3: contador privado + critical (sincroniza so uma vez no final)
void versao_privado_critical() {
    double inicio = omp_get_wtime();
    long count = 0;

    #pragma omp parallel
    {
        unsigned int seed = 12345 + omp_get_thread_num();
        long count_local = 0; // cada thread conta sozinha aqui, sem
sincronizar nada

```

```

        #pragma omp for
        for (long i = 0; i < N; i++) {
            double x = (double)meu_rand(&seed) / 0x7fff;
            double y = (double)meu_rand(&seed) / 0x7fff;
            if (x*x + y*y <= 1.0) {
                count_local++; // sem sincronizacao nenhuma aqui dentro ->
rapido
            }
        }

        // so sincroniza UMA vez por thread, no final
        #pragma omp critical
        {
            count += count_local;
        }
    }

    double pi = (4.0 * count) / N;
    double fim = omp_get_wtime();
    printf("3) Privado + critical      -> pi: %.6f | Tempo: %fs\n", pi, fim -
inicio);
}

// VERSAO 4: contador privado + atomic (sincroniza so uma vez no final)
void versao_privado_atomic() {
    double inicio = omp_get_wtime();
    long count = 0;

    #pragma omp parallel
    {
        unsigned int seed = 12345 + omp_get_thread_num();
        long count_local = 0;

        #pragma omp for
        for (long i = 0; i < N; i++) {
            double x = (double)meu_rand(&seed) / 0x7fff;
            double y = (double)meu_rand(&seed) / 0x7fff;
            if (x*x + y*y <= 1.0) {
                count_local++;
            }
        }

        // junta o resultado local no compartilhado, uma vez por thread
        #pragma omp atomic
        count += count_local;
    }
}

```

```

    double pi = (4.0 * count) / N;
    double fim = omp_get_wtime();
    printf("4) Privado + atomic          -> pi: %.6f | Tempo: %fs\n", pi, fim -
inicio);
}

// VERSAO 5: clausula reduction (deixa o OpenMP cuidar de tudo sozinho)
void versao_reduction() {
    double inicio = omp_get_wtime();
    long count = 0;

    #pragma omp parallel
    {
        unsigned int seed = 12345 + omp_get_thread_num();

        // reduction(+:count) -> cada thread ganha uma copia local e o OpenMP
soma tudo no fim
        #pragma omp for reduction(+:count)
        for (long i = 0; i < N; i++) {
            double x = (double)meu_rand(&seed) / 0x7fff;
            double y = (double)meu_rand(&seed) / 0x7fff;
            if (x*x + y*y <= 1.0) {
                count++; // sem diretiva de sincronizacao nenhuma, o reduction
resolve
            }
        }
    }

    double pi = (4.0 * count) / N;
    double fim = omp_get_wtime();
    printf("5) Reduction                -> pi: %.6f | Tempo: %fs\n", pi, fim -
inicio);
}

int main() {
    printf("Estimativa de PI - Monte Carlo (N = %d pontos)\n", N);
    printf("Threads disponiveis: %d\n\n", omp_get_max_threads());

    versao_compartilhado_critical();
    versao_compartilhado_atomic();
    versao_privado_critical();
    versao_privado_atomic();
    versao_reduction();

    return 0;
}

```

8.2. OUTPUT

```
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa10> ./tarefa10
Estimativa de PI - Monte Carlo (N = 100000000 pontos)
Threads disponiveis: 16

1) Compartilhado + critical -> pi: 3.141446 | Tempo: 4.687000s
2) Compartilhado + atomic   -> pi: 3.141446 | Tempo: 1.418000s
3) Privado + critical       -> pi: 3.141446 | Tempo: 0.077000s
4) Privado + atomic        -> pi: 3.141446 | Tempo: 0.074000s
5) Reduction               -> pi: 3.141446 | Tempo: 0.086000s
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa10>
```